

CityWatch Performance & Storage Assessment

Architecture review of the seven production projects in `CityWatch.sln` , a ranked list of performance fixes per application, a plan for moving uploaded and generated files from the IIS VM to Azure Blob Storage, and a sprint timeline for doing all of it.

Date **9 October 2026** Codebase **master @ d9e35c72**
Scope **Web, Kpi, RadioCheck, Data, Common, Events, Tracking** (test projects excluded)
Prepared for **Globocon – CityWatch team**

01

Executive summary

~140k lines of C# across 7 projects, plus ~76k lines of page JS	150+ specific performance findings, each with a file and line reference
38 file-storage features found; 29 should move to Blob	7 sprints 19 Oct 2026 → 12 Feb 2027, about 670 dev hours + QA

What is slowing the system down

- Whole-table reads followed by in-memory filtering.** `GetClientSiteLogBooks()` loads every logbook to find a single row, and it is called inside per-logbook loops in the nightly site-log upload and the KVL KPI report.
`GetClientSiteRadioChecksActivityDetails()` loads the entire activity-status table, and one radio-check save does this 3–6 times. There are 54 more `_context.X.ToList()` full-table loads in the providers.
- Hot tables have no supporting indexes.** None of the 432 DbScripts indexes `GuardLogs.ClientSiteLogBookId` , `VehicleKeyLogs` (logbook, rego, entry time), `GuardLogins(ClientSiteId, LoginDate)` or `GuardLogsDocumentImages.GuardLogId` .
- Scheduled jobs and reports run per row.** Manning checks, radio-check clean-up and duplicate-login clean-up each run 6–8 queries plus a `SaveChanges` per row. Kpi reports re-read all guards four times per site and generate sites one after another.
- RadioCheck dashboards trigger server work on every page view.** Every page load calls the production status-recompute API through a new `HttpClient` . The page also reloads itself every 4 minutes, and the control-room map polls three full-table aggregates every 5 seconds for each operator.
- Charts launch a headless Chromium per image in Web.** Kpi already reuses one shared converter. Web does not, and also blocks on weather and UV HTTP calls without a timeout.
- Almost everything is synchronous.** The code has 470 sync `SaveChanges` against 16 async, ~38 sync-over-async calls, `Thread.Sleep(500)` for every offline record in the mobile API, and 50+ new SMTP connections opened per email.
- There is no caching.** `IMemoryCache` is used in one place. Lookups such as sites, guards, subdomains and KVL fields are re-read on every request, including by `_Layout` . None of the ~90 DataTables uses server-side paging, and every Web page ships about 9 MB of unminified JS.

Files on the VM

Uploads and generated files sit under each site's `wwwroot` . They are publicly served by `UseStaticFiles` , shared between apps through hard-coded paths such as `C:\c4isystem\Websites\kpi\...` and host URLs, and in four features found only by listing directories. Azure Blob is already used for IR PDFs (`irfiles`) and radio audio (`audiorecordings`) on account `c4istorage1` , but each call builds its own client. The

...additions / on account of storage, but each can take its own share. The recommended approach:

- Introduce one `IFileStorage` abstraction.
- Keep each file's relative path as its blob name, so most DB values (bare filenames) need no change.
- Add a legacy-URL redirect so existing links in JS, emails and DB rows keep working on day one.

Fix before anything else. Storage, Dropbox, SMTP, Google, AI and JotForm secrets are committed in `appsettings.json`. The storage account key is also hard-coded in `CityWatch.Data/Models/DailyPatrolData.cs:280`. RadioCheck keeps admin privilege flags in *static* fields, so one user's admin state leaks to every other user. See §7.

02

Architecture as it stands

Three ASP.NET Core 7 Razor Pages sites and four class libraries. All of it runs on one Windows/IIS VM with SQL Server 2022 on the same machine. Deployment is a manual Visual Studio folder publish followed by robocopy, and there is no CI/CD (`.github/workflows` is empty).

HOSTS (IIS SITES ON ONE VM)

CityWatch.Web 58k C# lines · 33 pages, ~596 handlers · 16 API controllers including the 6,858-line mobile API · SignalR · ShiftCancellation hosted service
CityWatch.Kpi 18k lines · KPI portal · scheduled PDF reports run inside HTTP GET (Task Scheduler) · FastReport background path · anonymous by folder
CityWatch.RadioCheck 21k lines · control-room dashboards, maps, JotForm webhooks · <i>references the Web project</i> · anonymous by folder

LIBRARIES

CityWatch.Data EF Core 7 DB-first · 219 DbSets · 17 providers (22.8k lines; GuardLogDataProvider alone is 9,455) · DbContext injects SignalR hub
CityWatch.Tracking 7k lines · live patrol tracking · own NoTracking DbContext, bounded channels, SqlBulkCopy, leader-gated workers. The cleanest design in the solution.
CityWatch.Common / Events Dropbox + SMS services, SignalR hub · in-process domain-event bus (single consumer channel)

EXTERNAL

SQL Server 2022 (same VM) One DB <code>prod-citywatch</code> ; triggers on GuardLogs, IncidentReports, GuardLogins
--

Storage

Local wwwroot folders · Dropbox (archive) · Azure Blob `c4istorage1` (irfiles, audiorecordings)

Services

SMTP (MailKit), SMSGlobal, Google Maps, open-meteo, ConvertAPI, Anthropic, LanguageTool, JotForm, FCM, Node/Chromium for charts

Project inventory

PROJECT	TYPE	C# LINES	KEY PACKAGES	NOTES
Web	Razor Pages + API	58,146	EF 7.0.15, iText7 7.2, Azure.Storage.Blobs 12.18, Dropbox 7, Jering NodeJS, Magick.NET, ConvertApi, Office Interop	Cookie auth; session in memory (120 min); 2 GB request limit; PowerPoint COM automation on the server (<code>hr.cs:2866</code>)
Kpi	Razor Pages + API	17,966	iText7, ClosedXML, Jering, MailKit, Dropbox	<code>AllowAnonymousToFolder("/");</code> ; report overlap guard commented out; no compression
RadioCheck	Razor Pages + API/webhooks	20,855	iText7, ClosedXML, EPPlus (HintPath to bin!)	References Web project; static admin flags; leftover FortescueWebApp demo code
Data	Class library	34,543	EF Core 7.0.15, Azure.Storage.Blobs	No pooling, tracking on by default, 5-min command timeout, fake trigger on every entity
Tracking	Feature library	6,988	EF Core, FirebaseAdmin	Gated by <code>Tracking:Enabled</code> ; <code>[Authorize]</code> on its APIs
Events	Class library	806	MS.Extensions abstractions	Only active when Tracking is on
Common	Class library	333	Dropbox, SMSGlobal, SignalR 1.1.0 (ASP.NET Core 2-era)	Dropbox client rebuilt and re-authenticated on every call (<code>.Result</code>)

Structural problems that make performance work harder

- **Duplicated logic across apps:** 3 ViewDataServices, 2 GuardLogReportGenerators, 3 Admin/Settings page models (4.8k / 3.1k / 0.8k lines), `LEDStatusForLoginUser` in 7 files. A fix must be applied to every copy.
- **God files:** `GuardLogDataProvider` 9,455 lines; `SaveClientSiteRadioCheckNew` is a single 1,742-line method with 62 `SaveChanges` calls.
- **Long jobs inside HTTP GETs** triggered by Windows Task Scheduler, with no queue, lock or retry.
- **DbContext coupled to SignalR:** every `SaveChanges` runs extra per-row queries and fires unawaited `SendAsync` . This also rules out `AddDbContextPool` .
- **12+ stored procedures and the `Insert_GuardLogs` trigger are not in source control**, so they cannot be reviewed.
- **Single-instance assumptions:** in-memory session, per-process SignalR with no backplane, and background services with no leader gating. Scaling out is not possible today.

03

Performance findings and fixes by application

Hours are development estimates for one experienced .NET developer and exclude QA. **HIGH** means a user-visible delay or a load multiplier. **MED** means measurable waste. **LOW** means hygiene. Line numbers refer to master @ d9e35c72.

Effort by application

AREA	MAIN THEMES	HIGH	MED/LOW	TOTAL H
CityWatch.Data (EF providers, DbContext)	Full-table loads, N+1 in scheduled jobs, SaveChanges in loops, split queries, AsNoTracking, async	55	65	120
SQL Server (indexes, procs, triggers)	Missing indexes, non-sargable timezone filters, trigger bugs, unscripted procs	20	15	35
CityWatch.Web	Report generators, Chromium per chart, mobile sync sleeps, bulk IR zip, JS payload, lookups	25	35	60
CityWatch.Kpi	Logbook loads, sequential sites, chart pipeline, job overlap, import chatter	33	12	45
CityWatch.RadioCheck	CallApi on page load, map polling, broadcast fan-out, duplicate dashboard queries	24	16	40
Tracking / Events / Common	Analytics caching, event bus head-of-line blocking, Dropbox client reuse	—	12	12
Cross-cutting	Caching service, email queue, HttpClientFactory, bundling, static cache headers, load testing	20	38	58
Performance total		177	193	370

CityWatch.Web

ISSUE	LOCATION	FIX	H
HIGH Entire logbook table loaded to find one row, inside the nightly per-logbook loop	GuardLogReportGenerator.cs:113,1612,1809 , KeyVehicleLogReportGenerator.cs:71,83,97 , SiteLogUploadService.cs:121,152,799,933	GetClientSiteLogBookById(id) with includes; filter in SQL	3
HIGH New headless Chromium for every chart (1–3 s each, 5 per patrol PDF)	Scripts/ir-chart.js:51 , ir-uvchart.js:152 , effort-chart.js:125	Port Kpi's shared converter queue (Kpi/Scripts/ir-chart.js:15-31); return a buffer, not a file	3
HIGH IR generation blocks on weather/UV HTTP calls: new client, no timeout, no cache	IncidentReportGenerator.cs:290,1610,1856	Typed HttpClient with 5 s timeout; cache by rounded lat/lon + date; make the generator async	4
HIGH Thread.Sleep(500) per offline record (100 records ≈ 50 s blocked thread)	GuardSecurityNumberController.cs:1198-1238,5503-5526,5548-5568 , ScannerController.cs:155	Remove the sleeps, batch the inserts, send one SignalR refresh	3
HIGH Bulk IR zip downloads sequentially with a new HttpClient per file; date-named folder collides between users	PatrolData.cshtml.cs:2137-2185	Stream blobs into a ZipArchive 8 at a time; GUID work folder	3
HIGH ~9 MB of unminified JS on every page; no long-lived cache headers	Pages/Shared/_Layout.cshtml:502-507	Per-page scripts; lazy-load pdfmake/xlsx/html2pdf; minify; OnPrepareResponse Cache-Control with asp-append-version	10
MED DB queries in _Layout on every request; GetSubDomainDetails has 43 call sites	_Layout.cshtml:49,83 , DailyLog.cshtml.cs:105-109	IMemoryCache, 10–30 min TTL	2
MED All sites / guards / wands / KVL fields re-queried to find one row	ViewDataService.cs:667,905,3360,3476 ... ; PatrolData.cshtml.cs:1973-2319 (5× GetGuards)	Cached lookup service with invalidation on save; single-row queries	7
MED Lookups re-read inside report row loops; docket loads every POI profile; image query per log entry	IncidentReportGenerator.cs:948,1020,1069 ; KeyVehicleLogDocketGenerator.cs:103,141-166 ; GuardLogReportGenerator.cs:201,755,1227	Move the lookups out of the loops; query by IN (ids) ; use dictionaries	6
MED Timesheets run 9–10 single-value queries per guard	TimesheetReportGenerator.cs:312-336,407-419,506-518	One projection per guard set	3
MFD Zip and Dropbox work blocks request threads	AuditSiteLog.cshtml.cs:320.587.659.6	Asvnc handlers: background iob for large	5

LOW	Connect/auth/send/disconnect SMTP per email, inside requests (20+ sites)	RadioCheckNew.cshtml.cs:305... , Register.cshtml.cs:744 , KeyVehicleLog.cshtml.cs:1982 ...	Shared IEmailSender fed by a background Channel queue	8
MED	Mobile offline cache is copied per user×site; site log page size unclamped; notifications unpaged	GuardSecurityNumberController.cs:748-784,1900,5144	Global keys for shared lists; SizeLimit; clamp pageSize ≤ 50	4
MED	No server-side DataTables; logbook and KVL grids pull complete datasets	guardlog.js (18), keyvehiclelog.js (14), reports.js	serverSide:true with Skip/Take on the 6 heaviest grids	12
LOW	Shift-cancellation worker runs on every instance; 2 GB body limit; Console logging	ShiftCancellationEmailBackgroundService.cs , Program.cs:30,151	Leader flag; per-endpoint [RequestSizeLimit] ; ILogger	4

CityWatch.Kpi

ISSUE	LOCATION	FIX	H	
HIGH	Full logbook table loaded per logbook × per site (≈31 × sites per KVL report)	KeyVehicleGenerator.cs:154,699,763	Reuse the rows already fetched, or load by id	2
HIGH	Sites generated one after another in schedules, downloads and FastReport	SendScheduleService.cs:124-146,775-797 ; FastReportService.cs:177	Parallel.ForEachAsync with DOP 3–4, one DI scope per site (after the GUID chart names)	8
HIGH	~19 charts per site go PNG → disk → read → delete, blocking on task.Result	ReportGenerator.cs:846,2557-2603 ; Monthly :883 ; Weekly :722	Return bytes from Node, or embed SVG with iText SvgConverter ; async	12
HIGH	All guards loaded 4× per site; all sites loaded to read one State	ReportGenerator.cs:2057,2078,2098,2152,213 (and Monthly/Weekly)	Run-scoped cache (as FastReport already does)	4
HIGH	Chart temp names are second-resolution, so concurrent runs overwrite each other and catch{} hides it	ReportGenerator.cs:850,2570,2598	Guid.NewGuid() names; log failures	1
HIGH	Scheduler job can overlap itself (guard commented out) and runs inside a GET	API/KpiReportController.cs:43-49	sp_getapplock / lease row; hosted queue; return 202	5
HIGH	Run Now blocks the request thread for the whole multi-site run	Admin/Settings.cshtml.cs:1204,2339,2956	Async, routed through the FastReport job path	2
MED	Import job: 2 queries per day, a query per KPI row, ~90 Dropbox calls per site, new HttpClient	ImportDataService.cs:197-201,284-296,538,609-660	Grouped queries; .ToList() before the loop; cached Dropbox auth; HttpClientFactory	5
MED	Same report uploaded to Dropbox once per site, re-authenticating each time, whole file in memory	SendScheduleService.cs:295-372,836	Cache the team member; stream; de-duplicate paths	3
LOW	2.7 MB banner embedded in every summary; SmartWand settings queried twice; benchmark endpoint public	MonthlySummaryReportGenerator.cs:594 ; Settings.cshtml.cs:1363 ; FastKpiReportController.cs:158	Downscale and cache the image; delete the dead query; restrict the endpoint to admins	3

CityWatch.RadioCheck

	ISSUE	LOCATION	FIX	H
	HIGH Every dashboard load calls the <i>production</i> recompute API (hard-coded URL, new HttpClient, fire-and-forget); 4-min page reloads repeat it for every operator, and runs race	RadioCheckV2.cshtml.cs:97,2809 + 5 single pages; RadioChecksActivityStatusService.cs:42-221	Remove from page loads; one leader-gated <code>PeriodicTimer</code> (30–60 s); <code>ExecuteDelete</code>	6
	HIGH Control-room map polls 3 full-table aggregates every 5 s per operator, then refetches full lists	controlRoomMap.js:1384-1396 ; ControlRoomMap.cshtml.cs:304-345	Server-wide token cached for 3–5 s, or <code>SignalR</code> push; return deltas	8
	MEDIUM Main dashboard data table uses <code>CMATER</code> table, which is not indexed; 4-min page reloads repeat it for every operator, and runs race	controlRoomMap.js:1384-1396 ; ControlRoomMap.cshtml.cs:304-345	Cache dashboard data; use <code>SignalR</code> push; return deltas	10

MED	Connect/auth/send/disconnect SMTP per email, inside requests (20+ sites)	RadioCheckNew.cshtml.cs:305... , Register.cshtml.cs:744 , KeyVehicleLog.cshtml.cs:1982 ...	Shared IEmailSender fed by a background Channel queue	8
------------	--	--	---	---

HIGH	Nationwide broadcast: per-site log write + SMTP connection + full guard-log read inside one POST	RadioCheckV2.cshtml.cs:1289-1411 (×4 copies)	Set-based recipient query; bulk insert; queue email/SMS	10
MED	Active/inactive lists computed twice per load (counts, then DataTables); all guards loaded to find one	RadioCheckV2.cshtml.cs:143-198,376-441	Count queries; shared 5–10 s cache across operators	3
MED	4-min full page reload, 4 tables reloaded after every save, globe maps reload every 3 min	radiocheck.js:3,182,3089 ; globeMap.js:329	Targeted refreshes over the existing SignalR hub	8
MED	HttpClient created in controller constructors (webhooks, JotForm)	WebhookController.cs:158 , JotFormService.cs:35 ...	AddHttpClient<>	2
LOW	No compression; AddSession registered twice; 8 MB unused libDemo	Program.cs:65,75	Clean up the pipeline	3

Tracking, Events, Common

	ISSUE	LOCATION	FIX	H
MED	Analytics re-scan raw tables on every 60 s panel poll; Take(100) applied after materialising	Tracking/Api/AnalyticsController.cs:107-125,220-284,284	60 s IMemoryCache per window; GROUP BY and TOP in SQL	5
MED	Event bus: one consumer runs handlers sequentially; a slow handler causes silent DropOldest losses	Events/DomainEventDispatcher.cs:55-99	Parallel readers or Task.WhenAll ; alert on Metrics.Dropped	4
MED	Dropbox team client rebuilt and re-authenticated with .Result on every call	Common/Services/DropboxService.cs:28,47,105	Singleton client; cached member id; async all the way	3

04

Data layer, EF queries and SQL scripts

These measurements cover the providers plus the three ViewDataServices: .ToList() 1,162 · SaveChanges() 470 vs SaveChangesAsync 16 · AsNoTracking 40 against ~1,950 materialising calls · AsSplitQuery 2 · whole-table ToList 54 · no SQL-injection patterns found.

EF / provider fixes

ISSUE	LOCATION (CITYWATCH.DATA)	FIX	H	
HIGH	Whole <code>ClientSiteRadioChecksActivityStatus</code> table returned (tracked) to 45 callers that then filter in memory	<code>GuardLogDataProvider.cs:2253</code>	Filtered overload (guard, site) with <code>AsNoTracking</code> ; replace every call site	8
HIGH	Manning-check job: 6–8 queries plus <code>SaveChanges</code> per row; <code>RemoveRange</code> loads before deleting; 3 near-copies	:3294-3460, 3478, 3829	Prefetch sets per run; <code>ExecuteDelete</code> ; one <code>SaveChanges</code> ; merge the copies	10
HIGH	Radio-check clean-up loads the whole table, then re-queries and saves per row	:4023-4056	Single <code>ExecuteDelete</code> with NOT EXISTS	1.5
HIGH	Duplicate-login and 2-hour no-activity jobs: per-row queries and saves	:3973-4004, 7657-7700	Grouped load, <code>ToLookup</code> , batch save	5
HIGH	<code>GetActiveGuardDetails</code> re-runs the heavy procedure up to 5× when the result is legitimately empty; loads all wands and tags on each of ~25 callers	:2257-2400	Retry only on transient <code>SqlException</code> ; cache wand/tag lookups; short shared result cache	4
HIGH	Fusion logs: <code>GuardLogins</code> query has no site filter, $O(n \cdot m)$ <code>FirstOrDefault</code> per log	:7221-7242	Site filter; <code>ToDictionary</code> ; date range instead of <code>.Date</code>	2
HIGH	Guard logins: one query per guard over all history; <code>GuardLogins.ToList()</code> of the whole table	<code>GuardDataProvider.cs:432-458, 564-570</code>	One grouped <code>Max(Id)</code> query	2.5

HIGH	KVL save loads every historical KVL with the same rego (no index) and saves per row	GuardLogDataProvider.cs:1161-1190	ExecuteUpdate plus a VehicleRego index	1.5
HIGH	Every SaveChanges runs a query per changed GuardLog and fires unawaited SignalR sends	CityWatchDbContext.cs:74-82,156-206	One batched logbook→site lookup; awaited/logged send	2
MED	query.Include().Load(); return query.ToList() executes twice	GuardLogDataProvider.cs:979-1092 , IrDataProvider.cs:190-240	Run the query once	2
MED	Autocomplete uses Substring().ToLower() on every keystroke, scanning VehicleKeyLogs	:1656-1665, 1793-1810	StartsWith (CI collation), Take(20) , index	1.5
MED	56 .Date predicates on columns, 15 ToLower/ToUpper predicates	OnDuty.Date ×20, LoginDate.Date ×14 ...	Half-open ranges >= d && < d+1 ; drop the case functions	5
MED	Multi-collection Includes with no split query (cartesian); an unused 9-ThenInclude query	ClientDataProvider.cs:633 , ConfigDataProvider.cs:1631-1645 , KpiSchedulesDataProvider.cs:84-260	AsSplitQuery + projections; delete the dead query	3
MED	~15 N+1 loops in providers and ViewDataService (staff docs, holidays, audit history, duress links...)	see appendix lines M9–M20	Query with Contains , then ToLookup	10
MED	SaveChanges inside 19 loops. Two also reuse a tracked entity with Id=0 (bug when there is more than one item)	GuardLogDataProvider.cs:8006-8014, 8351-8358 ...	New entity per item; AddRange; one save	3
MED	SaveClientSiteRadioCheckNew :one 1,742-line method with 62 SaveChanges and no transaction	:4129	Status→action strategy; one unit of work	22
MED	Everything synchronous; almost no AsNoTracking	whole provider layer	Async plus AsNoTracking on hot read/write paths first	24
LOW	Fake trigger on every entity disables EF7's fast insert path; 5-min global timeout; no pooling or retry in Kpi/RC	CityWatchDbContext.cs:26,449,580-607	HasTrigger only on real trigger tables; 30–60 s default timeout; consistent options	3

SQL Server: indexes, procedures, triggers

	ISSUE	SCRIPT / OBJECT	FIX	H
HIGH	Hot tables have no nonclustered indexes in any of the 432 scripts	GuardLogs, VehicleKeyLogs, GuardLogins, GuardLogsDocumentImages, PatrolCarLogs, ClientSiteRadioChecks, IncidentReports, LoginUserHistory, HitLogs	New numbered script (below), built ONLINE, after checking sys.dm_db_missing_index_details and Query Store on production	6
HIGH	sp_GetActiveGuardDetailsForRC aggregates the whole GuardComplianceLicense table per call; CAST(AT TIME ZONE ...) = date join on HitLogs scans all history; HTML built in SQL	DbScript/320_Optimize_RadioCheck_Performance.sql:64-98,199-215	Limit the CTE to active guards; precompute UTC day bounds; HitUtcDateTime as an index key; move HTML to the UI	6
HIGH	Tag-scan summary converts every hit's timezone before filtering; correlated MyScans subquery	337_...Sp_GetClientSiteTagScanSummary.sql	Convert the parameters to UTC once; use the grouped COUNT	3
MED	sp_GetSiteLog ignores @LastLogId , so every poll returns the whole day; FORMAT() per row	330_...sp_GetSiteLog_LinkedSites.sql:8 , 380	Honour Id > @LastLogId ; CONVERT(char(5),...,108) ; temp tables	1.5
MED	IncidentReports trigger uses SELECT MAX(Id) instead of inserted (a race, and wrong for multi-row inserts)	165_Create_Trigger_Insert_IncidentReports.sql	Insert from inserted ; SET NOCOUNT ON	0.5
MED	SmartWand trigger fires on every GuardLogin UPDATE, so its table grows without limit	335_...trg_GuardLogin_SmartWandUse.sql	INSERT only, or only when SmartWandId changes	0.5
MED	Insert_GuardLogs trigger and 12+ procedures (e.g. MobileAppSp_GetSiteTagStatus , sp_GetActiveGuardSWDetailsForRC) are not scripted	production only	Script them from production into DbScript, then review	3

Proposed index script (next free DbScript number). Validate each index against production DMVs before applying, because some may already exist outside source control:

```
CREATE INDEX IX_GuardLogs_LogBook ON GuardLogs (ClientSiteLogBookId, EventDateTimeLocal DESC, Id DESC)
INCLUDE (GuardLoginId, IrEntryType, IsSystemEntry) WITH (ONLINE = ON);
CREATE INDEX IX_GuardLogs_GuardLogin ON GuardLogs (GuardLoginId) WITH (ONLINE = ON);
CREATE INDEX IX_VKL_LogBook ON VehicleKeyLogs (ClientSiteLogBookId, EntryTime) WITH (ONLINE = ON);
CREATE INDEX IX_VKL_Rego ON VehicleKeyLogs (VehicleRego) INCLUDE (EntryTime, ExitTime, HasLoadVariation);
CREATE INDEX IX_GuardLogins_Site_Date ON GuardLogins (ClientSiteId, LoginDate);
CREATE INDEX IX_GuardLogins_LogBook ON GuardLogins (ClientSiteLogBookId);
CREATE INDEX IX_GLDocImages_GuardLog ON GuardLogsDocumentImages (GuardLogId);
CREATE INDEX IX_PatrolCarLogs_LogBook ON PatrolCarLogs (ClientSiteLogBookId);
CREATE INDEX IX_CSRC_Site_Guard ON ClientSiteRadioChecks (ClientSiteId, GuardId);
CREATE INDEX IX_CSRCAS_Guard_Notif ON ClientSiteRadioChecksActivityStatus (GuardId, NotificationType)
INCLUDE (GuardLoginTime, ClientSiteId);
CREATE INDEX IX_HitLogs_Site_Time ON ClientSiteSmartWandTagsHitLogs (LoggedInClientSiteId, HitUtcDateTime) INCLUDE
(TagUId);
CREATE INDEX IX_HitLogs_LinkedSite_Time ON ClientSiteSmartWandTagsHitLogs (TagLinkedClientSiteId, HitUtcDateTime);
CREATE INDEX IX_LoginUserHistory_User ON LoginUserHistory (LoginUserId, LoginTime DESC);
```

05

Moving files from the VM to Azure Blob Storage

The code writes files to disk in 38 distinct features across Web, Kpi and RadioCheck. 29 hold permanent or shareable data and should move to Blob. 9 are scratch files (chart PNGs, map cache, PDF staging) and should stay in local temp storage, with only the finished output uploaded.

Current state

- **Already on Blob:** IR PDFs and oversized guard-log PDFs in `irfiles/{yyyyMMdd}/` ; radio audio in `audiorecordings/` . URLs to `c4istorage1.blob.core.windows.net/irfiles` are hard-coded in 15 places, including JS.
- **Dropbox** remains the archive for IR, site-log, docket, licence, certificate and KPI PDFs. Keep it as a secondary copy.
- **All three apps** serve uploads with a plain `UseStaticFiles()` , so anyone with a URL can read licences, KVL photos and staff documents.
- **Files are shared across apps through disk paths and host URLs:**
 - Web writes into the Kpi site through `C:\c4isystem\Websites\kpi\...` (`Settings.cshtml.cs:1515`).
 - Kpi reads a `Kv1Uploads` folder that does not exist in its own `wwwroot`.
 - RadioCheck links to `cws-ir.com/StaffDocs` .
 - Web and RadioCheck fetch `SiteImage` over HTTP from the Kpi host.
- **Volumes on the dev copy:** Web `wwwroot` 1.2 GB (`TA/AllTemplates.zip` alone is 958 MB), Uploads 65 MB, DglUploads 14 MB, RadioCheck `jotform` 55 MB. *Production volumes must be measured on the VM.*

Inventory: what moves, where to, and how the DB value is handled

FEATURE	APP	CURRENT FOLDER	DB COLUMN · STORED VALUE	TARGET CONTAINER / PREFIX	DB ACTION
Guard licence, compliance, HR docs	W	Uploads/Guards/Licence/{lic}/	GuardLicences / GuardCompliances / GuardComplianceLicense .FileName · bare name	guard-docs/License/{lic}/	None
Certificate &	W	.../License/{lic}/Cert	TrainingCourseCertificateRPL.FileName ·	guard-docs/... (same sub-path)	None

RPL docs, generated certificates		ificateDocuments RPL... /{course}/	bare		
KVL attachments, vehicle & person images, compliance docs	W (Kpi reads)	KvlUploads/{irego id} / , KvlUploads/Person/	No row (directory listing); POIImage flag	kvl-uploads/{irego id}/...	None; listing becomes GetBlobsByHierarchy(prefix)
Daily guard log images / videos	W + mobile API	DglUploads/{GuardLogId}/{Rear 25% Other}Files/	GuardLogsDocumentImages.ImagePath · absolute URL https://cws-ir.com/DglUploads/...	dgl-uploads/{GuardLogId}/...	Normalise to relative key (script below)
IR attachments (staging until PDF)	W + mobile	Uploads/{ReportReference}/	None (folder scan)	ir-attachments/{iref}/ , 30-day lifecycle	None
IR PDFs	W	Pdf/Output → ToDropbox → Archive	IncidentReports.FileName · bare	irfiles/{yyyyMMdd}/ (exists)	None; replace the hard-coded URLs
Staff docs / SOPs (separate copies in W and Kpi)	W, Kpi	StaffDocs/	StaffDocuments.FileName · bare; .FilePath · absolute base URL	staff-docs/ (one shared copy)	Clear FilePath; de-duplicate the two folders
Training courses & certificate templates, TA feedback	W	TA/{HRef}/Course Certificate/ , TA/Feedback/	TrainingCourses / TrainingCourseCertificate.FileName · bare	training/... ; chunked upload via staged blocks	None
IR form templates	W	Pdf/Template/	ReportTemplates.FileName/.Path · bare	templates/	None
Subdomain, company & PDF logos	W (writes into Kpi)	SubdomainLogo/ , Images/cr_*.JPG , CWSLogoPdf.png	SubDomain.Logo · bare; CompanyDetails.*LogoPath · absolute URL	branding/...	Normalise the logo paths; removes the cross-site write
Client site image, summary image	Kpi (W, RC read)	SiteImage/ , SummaryImage/	ClientSiteKpiSettings.SiteImage · /SiteImage/x ; KpiSendScheduleSummaryImages.FileName	branding/site-images/ , branding/summary/	Strip the leading /SiteImage/
RC action-list images	RC, Kpi (W reads)	RCImage/ or configured absolute folder	RCActionList.Imagepath · bare	rc-images/	None; remove the two folder settings
Client site key images	Kpi	KeyImage/{siteId}/	ClientSiteKeys.ImagePath · absolute ; ImagePathNew · bare	site-keys/{siteId}/	Rebuild ImagePath from ImagePathNew
Roster covers	W	Uploads/RosterCovers /	RosterGroups/RosterBinders.CoverFileName · bare	roster-covers/	None
Duress app audio / multimedia	W	DuressAppAudio/ , DuressAppMultimedia /	DuressAppField.Name · bare	duress-media/ (public-read or CDN; consumed by the mobile app)	None
Mobile app binaries, welcome pack	W	Downloads/MobileApp / , WelcomePack/	MobileAppUpgrade.FileName · bare	mobile-apps/ , public-assets/	None
Radio audio recordings	RC	AudioRecordings/{dd_MM_YYYY}/ (local copy	AudioRecordingLog.BlobUrl · full blob URL	audiorecordings/ (exists)	Stop writing the local copy

never deleted)						
JotForm submissions, Excel outputs	RC	uploads/jotform/{Form}/{WorkOrder}/	None	jotform/...		None
Strings / Quarantined register Excel (edited in place)	RC	uploads/jotform/StringsData/*.xlsx	None	jotform/registers/ or move to DB tables	with ETag/lease,	Decision needed
Generated outputs: site-log, KVL, docket, timesheet, wand, KPI PDFs; Excel; audit zips	W, Kpi, RC	Pdf/Output , Pdf/FromDropbox , Excel/Output	ClientSiteLogBooks.FileName, KeyVehicleLogDocketHistory.FileName , bare	reports/{yyyyMMdd}/ + downloads/ (7–30-day lifecycle, SAS links)		None. Fixes the missing FromDropbox cleanup
Stay local (scratch only): GraphImage chart PNGs, GpsImage / GpsImageLive map cache, %TEMP% IR composites, chunk staging, zip work folders, logs (→ Application Insights), static lib/css/js/images . Review: TA/AllTemplates.zip (958 MB) should move to training/ or be deleted if unused.						

Design principles

- Same relative path becomes the blob name.** wwwroot/KvlUploads/ABC123/photo.jpg → container kvl-uploads , blob ABC123/photo.jpg . Every column that stores a bare filename (most of them) keeps working without a data change.
- Private containers, served through the app.** Add one endpoint, /files/{container}/{**path} . It checks the user's access, then either 302-redirects to a 5–15 minute read-only SAS (best for large files) or streams the blob. Use public-read only for media the mobile app fetches anonymously (duress audio), and preferably put that behind Azure Front Door/CDN.
- A compatibility shim, so nothing breaks on cut-over day.** A small middleware placed before UseStaticFiles maps legacy prefixes (/KvlUploads/ , /DglUploads/ , /StaffDocs/ , /Uploads/ , /TA/ , /SubdomainLogo/ , /SiteImage/ , /RCImage/ , /KeyImage/ , /DuressApp*/ , /Pdf/) to the /files endpoint. Old links in JS, emails, PDFs and DB absolute URLs on cws-ir.com keep resolving. JS can then be cleaned up gradually.
- Generators keep a local temp workspace.** iText PdfWriter(path) , ZipFile, Node charts and ConvertAPI all need a path. They write to a per-job folder under Path.GetTempPath() , upload the result, and delete the folder. Merging, zipping and downloading should stream directly from Blob where possible.
- Authentication:** if the VM is an Azure VM, use Managed Identity with DefaultAzureCredential and the *Storage Blob Data Contributor* role. Otherwise read the connection string from Key Vault. Use one BlobServiceClient singleton per app. Rotate the existing c4istorage1 key, because it is in git.
- Storage settings:** soft delete (14 days) plus versioning on document containers; Cool tier after 90 days for irfiles , reports and dgl-uploads ; lifecycle deletion for downloads and ir-attachments .

Code changes for new files

Add one abstraction in CityWatch.Common with two implementations, registered by configuration (local for developers, Blob for test and production):

```
public interface IFileStorage
{
    Task SaveAsync(string container, string path, Stream content, string contentType, CancellationToken ct = default);
    Task<Stream> OpenReadAsync(string container, string path, CancellationToken ct = default);
    Task<bool> ExistsAsync(string container, string path, CancellationToken ct = default);
    Task DeleteAsync(string container, string path, CancellationToken ct = default);
    Task DeletePrefixAsync(string container, string prefix, CancellationToken ct = default); // replaces
Directory.Delete
    IAsyncEnumerable<string> ListAsync(string container, string prefix, CancellationToken ct = default); // replaces
Directory.GetFiles
    Task CopyPrefixAsync(string container, string fromPrefix, string toPrefix, CancellationToken ct = default); // KVL
rego-id moves
    Uri GetReadUrl(string container, string path, TimeSpan ttl); // short SAS (user delegation key)
}
```

```
// Program.cs (Web, Kpi, RadioCheck)
builder.Services.Configure<FileStorageOptions>(builder.Configuration.GetSection("FileStorage"));
builder.Services.AddSingleton(sp => new BlobServiceClient(new Uri(opts.AccountUrl), new DefaultAzureCredential()));
builder.Services.AddSingleton<IFileStorage, AzureBlobFileStorage>(); // LocalFileStorage when Provider = "Local"
app.UseMiddleware<LegacyFilePathRedirectMiddleware>(); // before UseStaticFiles()

// Typical call-site change (KeyVehicleLog.cshtml.cs OnPostUpload)
- using var fs = System.IO.File.Create(Path.Combine(_env.WebRootPath, "Kv1Uploads", rego, file.FileName));
- file.CopyTo(fs);
+ await using var s = file.OpenReadStream();
+ await _files.SaveAsync("kv1-uploads", $"{rego}/{file.FileName}", s, file.ContentType);
```

- Replace the 6 inline `new BlobServiceClient(...)` uses, `DownloadPDF.cshtml.cs` (whose container comes from the query string, which must be fixed) and every hard-coded `c4istorage1.../irfiles` URL with `IFileStorage / /files/irfiles/...`
- Add a `FileUrlHelper.For(container, path)` for Razor and JSON so the server builds URLs instead of JS string concatenation. Expose it to JS through a `data-files-base` attribute.
- Remove the cross-app paths: `KpiWebUrl + SiteImage`, the hard-coded Kpi path in Web Settings, and `RCActionListKpiImageFolder / WebActionListKpiImageFolder`. All apps read the same containers.
- Mobile API: `GuardSecurityNumberController` upload methods write relative keys (`dgl-uploads/{id}/RearFiles/x.jpg`) rather than `https://cws-ir.com/DglUploads/...`
- Add `[RequestSizeLimit]` per upload endpoint, validate content type and extension, and drop the 2 GB global limit.

Migrating existing files and the paths in the DB

1. **Inventory on the VM.** Measure file counts and sizes per folder for each site (`C:\c4isystem\Websites\{ir,kpi,rc}\prod-citywatch\wwwroot`), then compare DB rows with files on disk to list orphans and missing files.
2. **Bulk copy with AzCopy** (no downtime; repeatable). Run an initial copy, then `azcopy sync` nightly until cut-over:

```
azcopy copy "C:\c4isystem\Websites\ir\prod-citywatch\wwwroot\Kv1Uploads\*"
"https://c4istorage1.blob.core.windows.net/kv1-uploads" --recursive --put-md5
azcopy sync "C:\c4isystem\Websites\ir\prod-citywatch\wwwroot\DglUploads"
"https://c4istorage1.blob.core.windows.net/dgl-uploads" --recursive --delete-destination=false
```

3. **Normalise absolute URLs in the DB** with a numbered DbScript that first copies the old values to a backup table:

```
SELECT Id, ImagePath INTO _bak_GuardLogsDocumentImages_ImagePath FROM GuardLogsDocumentImages WHERE ImagePath
LIKE 'http%';
UPDATE GuardLogsDocumentImages
SET ImagePath = SUBSTRING(ImagePath, CHARINDEX('/DglUploads/', ImagePath) + 12, 4000) -- →
'{id}/RearFiles/x.jpg'
WHERE ImagePath LIKE 'http%/DglUploads/%';
-- same pattern: ClientSiteKeys.ImagePath, StaffDocuments.FilePath, CompanyDetails.*LogoPath,
ClientSiteKpiSettings.SiteImage
```

Readers accept both forms during the transition: `value.StartsWith("http") ? legacy : FileUrlHelper.For(...)`.

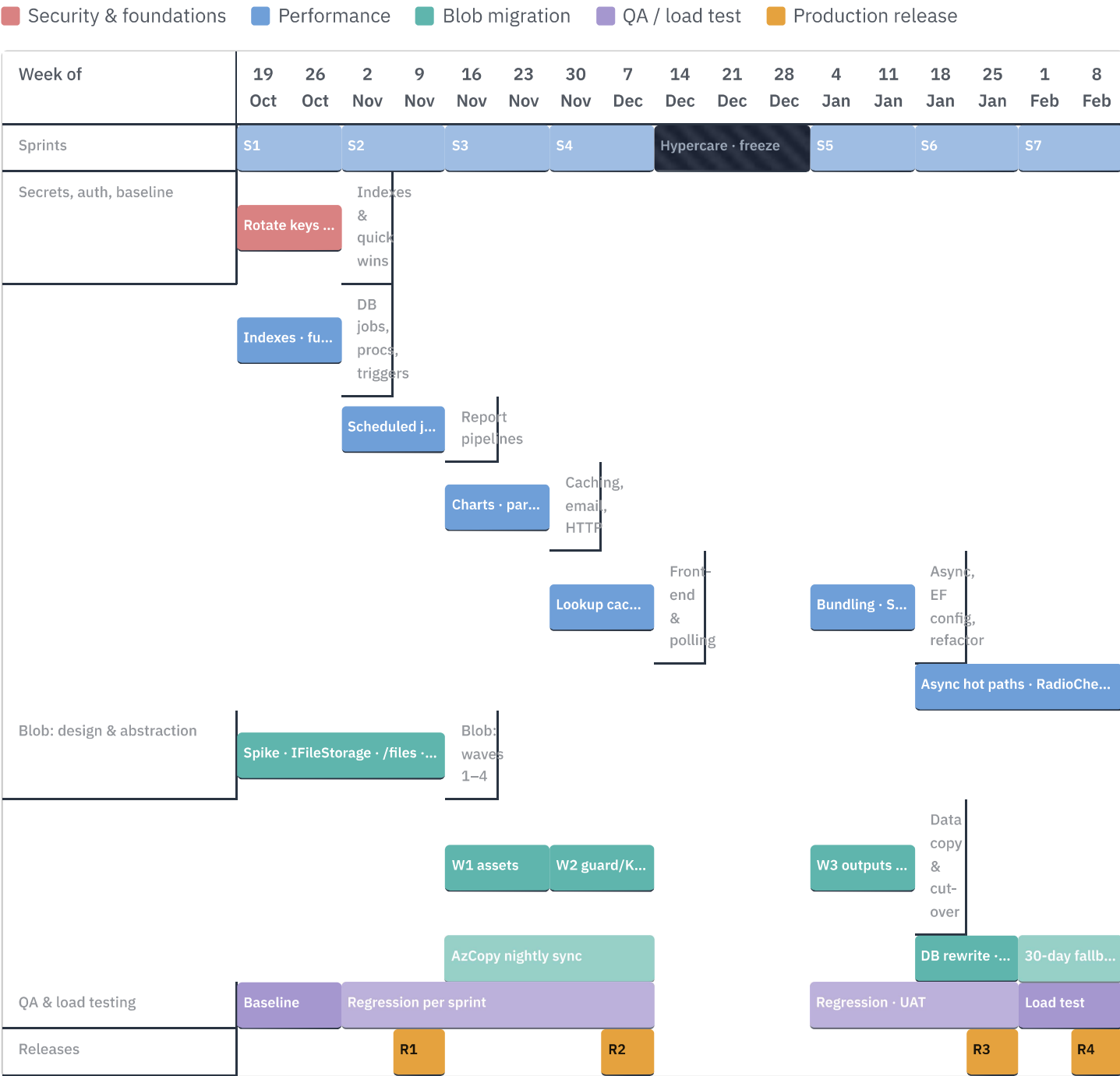
4. **Cut-over per wave, behind a feature flag** (`FileStorage:Provider` plus a per-feature switch): final `azcopy sync`, then flip the flag, then the new code writes to Blob. For 30 days, reads fall back to local disk if a blob is missing.
5. **Verification job:** compare blob count and MD5 against disk for each container, and confirm every DB-referenced file resolves.
6. **Decommission:** after 30 clean days, archive the local folders, remove them from the robocopy `/XD` exclusions, and drop the fallback code.

Recommended wave order.

1. Admin-managed assets: logos, templates, staff docs, training, site/summary/RC/key images, duress media, mobile apps. Low traffic; removes the cross-app paths.
2. Guard documents, KVL and DGL uploads: highest value, and they include the directory listing and absolute-URL rows.
3. Generated outputs and IR staging: removes local accumulation (`FromDropbox` has no clean-up).
4. RadioCheck JotForm and audio, plus a decision on the Excel registers.

Sprint plan and timeline

7 two-week sprints from **Mon 19 Oct 2026** to **Fri 12 Feb 2027**, with a change freeze over Christmas and New Year. Team: 2 .NET developers (one leads performance, one leads storage; both swap as needed), 1 QA at 50%, and DBA/infra support for indexes, Azure RBAC and AzCopy. Capacity is about 60 focused dev hours per developer per sprint.



Sprint 1 · Foundations & quick wins

84 h

19 Oct – 30 Oct 2026

- SEC** Rotate all committed secrets; move them to Key Vault / user-secrets; fix RadioCheck static admin flags; [Authorize] or key on report, cleanup and sync endpoints
- PERF** Enable Query Store, add Application Insights, record baseline timings for the top 15 pages, reports and jobs
- PERF** Index script (validated on production); remove full logbook loads (Web, Kpi, RC); remove CallApi from RC page loads in favour of a hosted timer; remove Thread.Sleep ; GUID chart names; stop retrying on empty results
- BLOB** Design spike: container map, auth model, VM inventory script

Sprint 2 · Database hot paths

110 h

2 Nov – 13 Nov · **Release R1**

- **PERF** Filtered radio-check activity query (45 callers); manning, clean-up, duplicate-login and no-activity jobs set-based; KVL save `ExecuteUpdate` ; `DbContext SaveChanges` hook batched
- **PERF** Fix `sp_GetActiveGuardDetailsForRC` , the tag-scan summary, `sp_GetSiteLog` and both triggers; script the missing procs and `Insert_GuardLogs` into `DbScript`
- **PERF** Double-execution Include pattern, autocomplete `StartsWith` , split queries, `ViewDataService N+1s`
- **BLOB** `IFileStorage` (Local + Azure), `/files` endpoint with SAS, legacy-path middleware, containers, RBAC, lifecycle rules

Sprint 3 · Report pipelines

110 h

16 Nov – 27 Nov

- **PERF** Web shared Chromium converter; Kpi charts in memory or SVG; run-scoped lookup cache; parallel per-site generation; Kpi job lock plus background queue; async Run Now
- **PERF** Report N+1s (docket, guard-log images, timesheet, IR lookups); Kpi import job; Dropbox client reuse and streaming
- **BLOB** Wave 1: logos, templates, staff docs (de-duplicated), training, site/summary/RC/key images, duress media, mobile apps. Start the nightly AzCopy sync

Sprint 4 · Caching & integrations

110 h

30 Nov – 11 Dec · **Release R2** (Blob W1 live, W2 behind a flag)

- **PERF** Cached lookup service; `_Layout` cache; `ISender` plus background queue; `HttpClientFactory` everywhere; weather and geocode cache; RC broadcast fan-out set-based; mobile cache keys and paging
- **BLOB** Wave 2: guard licence/compliance/certificates, KVL uploads (prefix listing, `rego`→`id` copy), DGL uploads (mobile API writes relative keys), IR attachment staging

Hypercare & holiday freeze

—

14 Dec 2026 – 1 Jan 2027

- Monitor R2 in production against the baseline; fix defects only; no feature releases
- AzCopy sync keeps running; reconciliation reports reviewed

Sprint 5 · Front-end & outputs

95 h

4 Jan – 15 Jan 2027

- **PERF** JS bundling, minification and lazy libraries; static cache headers; compression in Kpi and RC; RC `ChangeToken` cache / SignalR push; replace the 4-min and 3-min page reloads; `SaveChanges`-in-loop bug fixes
- **BLOB** Wave 3: all generated PDFs, Excel and zips to `reports` and `downloads` (SAS, lifecycle), with hard-coded `irfiles` URLs replaced. Wave 4: RC JotForm and audio; register Excel decision

Sprint 6 · Cut-over

90 h

18 Jan – 29 Jan · **Release R3** (Blob cut-over)

- **BLOB** DB path normalisation scripts with backups; final sync; flag flip per wave; verification job; UAT sign-off

- **PERF** Server-side DataTables for the 6 heaviest grids; async on the hot logbook, KVL and RC paths; AsNoTracking sweep; EF options (timeout, retry, real triggers only); Tracking analytics cache

Sprint 7 · Hardening

72 h

1 Feb – 12 Feb · **Release R4**

- **PERF** Refactor `SaveClientSiteRadioCheckNew` into one unit of work; parallel event-bus dispatch
- **PERF** Load test and before/after report against the S1 baseline; regression fixes
- **BLOB** Decommission local folders after 30 clean days (end of Feb); remove the fallback code; runbook and handover docs

Effort summary

WORKSTREAM	S1	S2	S3	S4	S5	S6	S7	TOTAL
Security & foundations	42	—	—	—	—	—	—	42
Performance	30	70	60	50	50	50	60	370
Blob migration	12	40	50	60	45	40	12	259
Development	84	110	110	110	95	90	72	671
QA (≈20%) · DBA/infra support	regression every sprint, UAT in S6, load test in S7 · index builds, RBAC, Key Vault, AzCopy							~135 + ~30

Expected outcomes (to be confirmed against the Sprint 1 baseline)

- Logbook and KVL page loads and the site-log poll: index seeks instead of scans. Expect a large drop in DB CPU and reads on GuardLogs and VehicleKeyLogs.
- Nightly site-log upload and KPI schedules: the full-table loads per logbook are removed and sites run in parallel, so expect run time to drop by a multiple rather than a percentage.
- IR and patrol PDFs: no Chromium launch per chart and no blocking weather calls, saving several seconds per report.
- RadioCheck: server work no longer scales with the number of operators; one recompute per interval instead of one per page view.
- Web pages: about 9 MB of JS cut to only what each page needs, cached long-term after the first visit.
- VM disk: user data off the VM; deployments no longer copy or exclude upload folders; files are private by default; apps can be scaled out later.

Risks and dependencies

RISK	MITIGATION
No automated test coverage for the UI and reports; logic is copied across apps	Golden-file PDF comparisons for the main reports before refactoring; a checklist of sibling copies for each fix
Production schema differs from DbScript (unscripted procs and indexes)	Script production objects in S1/S2 before changing them; check DMVs before adding indexes
Manual IIS deployment with no CI	Optional: add a GitHub Actions build plus publish artefact (≈16 h, not included in the totals)
Production file volume and orphan count unknown	VM inventory in S1 refines the Wave 2–3 estimates; AzCopy runs as a background sync
Mobile app builds embed absolute file URLs	The legacy-path shim keeps <code>cws-ir.com/DglUploads/...</code> working; change the URL format only in the next app release
Strings register Excel is edited concurrently	Blob ETag/lease in the interim; recommend moving it to DB tables (separate estimate)

Security issues found during the review

These were outside the brief but are serious enough to schedule in Sprint 1.

FINDING	WHERE
HIGH Live secrets committed to git: SMTP password, Dropbox tokens and app secret, Google, AI and JotForm keys, Azure storage key, Wand token. The storage key is also hard-coded in a model class.	{Web,Kpi,RadioCheck}/appsettings.json ; Data/Models/DailyPatrolData.cs:280
HIGH Process-wide static admin flags, so one user's privilege state applies to every other user	RadioCheck/Helpers/AuthUserHelper.cs (set in ActiveGuardSinglePage.cshtml.cs:173- 201)
HIGH No API controller in Web, Kpi or RadioCheck has [Authorize] (report triggers, cleanup, sync, webhooks); Kpi and RC allow anonymous access to all pages	API/* ; Kpi/Program.cs , RadioCheck/Program.cs
HIGH Uploaded licences, KVL photos and staff documents are publicly readable through static files	UseStaticFiles() over wwwroot upload folders. Fixed by the Blob migration
MED Blob download page takes the container name from the query string	Web/Pages/DownloadPDF.cshtml.cs:23
MED Hard-coded static encryption key; server-side PowerPoint COM automation; 2 GB request limits	Data/Helpers/PasswordHelper.cs:11 ; Web/Pages/Admin/hr.cs:2866 ; Web/Program.cs:151

Method and assumptions

- A static review of source at master @ d9e35c72. No code was run against production and no changes were made. Findings come from reading the code; their impact must be confirmed with the Sprint 1 baseline (Query Store, Application Insights, report timings).
- Index recommendations are based only on DbScript. Production may already have some of these indexes, so validate each against sys.indexes and the missing-index DMVs first.
- Folder sizes come from the developer copy. Production volumes may be much larger and could change the AzCopy timing, though not the approach.
- Estimates are for experienced .NET developers familiar with the codebase, include unit-level testing and code review, and exclude QA, UAT coordination and CI/CD set-up. Treat the totals as ±20%.
- Azure: assumes the existing c4istorage1 account (Australia region) is reused and that the hosting VM can use a Managed Identity or Key Vault.

